Message sprint 2 Channels dms composer

MESSAGING SPRINT 2 — CHANNELS + DMs + COMPOSER

PREREQUISITE: Messaging Sprint 1 complete

PASTE THIS INTO REPLIT AI AGENT

WHAT TO BUILD

1. Channel CRUD tRPC router (create, list, join, leave, archive, browse, settings)
2. DM/Conversation tRPC router (create DM, create group, list conversations)
3. Message Composer component (rich text, @mentions, emoji, slash commands, file upload)
4. Channel Sidebar component (channel list with unread badges, DMs, sections)
5. Message List component (virtual scroll, message grouping, date separators)
6. Message Item component (hover actions, edit mode, delete, reactions)
7. Main Messaging Layout (3-column responsive)

Verify after:

- Create a channel → appears in sidebar
- Click channel → messages load in main panel
- Type message → send → appears for all members in real-time
- @mention a user → autocomplete dropdown → notification sent

- Start a DM → conversation created → can exchange messages
- Rich text: bold, italic, code, quote, list all work

FEATURE: CHANNEL SYSTEM

tRPC Router: `src/server/routers/channels.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { getIO } from "@/server/services/socket-server";

export const channelsRouter = router({
  // List channels user is a member of (for sidebar)
  listMy: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      const memberships = await prisma.channelMember.findMany
({
        where: { userId: ctx.user.id },
        include: {
          channel: {
            select: {
              id: true, name: true, displayName: true, type:
true, icon: true, color: true,
              isArchived: true, isPinned: true, lastMessageA
t: true, lastMessagePreview: true,
              memberCount: true, topic: true,
            },
          },
        },
        orderBy: { channel: { lastMessageAt: "desc" } },
      });

      return memberships
```

```
        .filter(m => !m.channel.isArchived)
        .map(m => ({
          ...m.channel,
          isStarred: m.isStarred,
          isMuted: m.isMuted,
          notificationPref: m.notificationPref,
          sidebarSection: m.sidebarSection,
          role: m.role,
        }));
    }),

  // Browse all public channels (for channel browser)
  browse: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      search: z.string().optional(),
      type: z.string().optional(),
      sortBy: z.enum(["name", "memberCount", "lastMessageA
t"]).default("memberCount"),
    }))
    .query(async ({ input, ctx }) => {
      const channels = await prisma.channel.findMany({
        where: {
          workspaceId: input.workspaceId,
          type: { in: ["public", "social"] }, // only browsea
ble types
          isArchived: false,
          deletedAt: null,
          ...(input.search ? { displayName: { contains: inpu
t.search, mode: "insensitive" } } : {}),
          ...(input.type ? { type: input.type } : {}),
        },
        orderBy: input.sortBy === "name"
          ? { displayName: "asc" }
          : input.sortBy === "memberCount"
          ? { memberCount: "desc" }
```

```
          : { lastMessageAt: "desc" },
      });

      // Check which channels user is already a member of
      const myMemberships = await prisma.channelMember.findMa
ny({
        where: { userId: ctx.user.id, channelId: { in: channe
ls.map(c => c.id) } },
        select: { channelId: true },
      });
      const memberOfSet = new Set(myMemberships.map(m => m.ch
annelId));

      return channels.map(c => ({ ...c, isMember: memberOfSe
t.has(c.id) }));
    }),

  // Get channel details
  getById: protectedProcedure
    .input(z.object({ channelId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      const channel = await prisma.channel.findUnique({
        where: { id: input.channelId },
        include: {
          members: {
            include: { channel: false },
            orderBy: { joinedAt: "asc" },
          },
        },
      });
      if (!channel) throw new Error("Channel not found");

      const membership = channel.members.find(m => m.userId =
== ctx.user.id);
      return { ...channel, myRole: membership?.role, isMembe
r: !!membership };
```

```
    }),

  // Create channel
  create: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      name: z.string().min(1).max(80).regex(/^[a-z0-9-]+$/,
"Only lowercase letters, numbers, and hyphens"),
      displayName: z.string().min(1).max(80),
      description: z.string().max(500).optional(),
      type: z.enum(["public", "private", "project", "team",
"announcement", "social"]).default("public"),
      icon: z.string().optional(),
      color: z.string().optional(),
      isDefault: z.boolean().default(false),
      linkedProjectId: z.string().uuid().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      // Check for duplicate name
      const existing = await prisma.channel.findUnique({
        where: { workspaceId_name: { workspaceId: input.works
paceId, name: input.name } },
      });
      if (existing) throw new Error(`Channel #${input.name} a
lready exists`);

      const channel = await prisma.channel.create({
        data: {
          ...input,
          slug: input.name,
          createdBy: ctx.user.id,
          memberCount: 1,
        },
      });

      // Add creator as owner
```

```
      await prisma.channelMember.create({
        data: { channelId: channel.id, userId: ctx.user.id, r
ole: "owner" },
      });

      // Post system message
      await prisma.message.create({
        data: {
          channelId: channel.id,
          authorId: ctx.user.id,
          content: `created the channel #${channel.displayNam
e}`,
          type: "system",
        },
      });

      return channel;
    }),

  // Join a channel
  join: protectedProcedure
    .input(z.object({ channelId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const channel = await prisma.channel.findUnique({ wher
e: { id: input.channelId } });
      if (!channel) throw new Error("Channel not found");
      if (channel.type === "private") throw new Error("Cannot
join a private channel — you must be invited");

      const existing = await prisma.channelMember.findUnique
({
        where: { channelId_userId: { channelId: input.channel
Id, userId: ctx.user.id } },
      });
      if (existing) return existing;
```

```
      const member = await prisma.channelMember.create({
        data: { channelId: input.channelId, userId: ctx.user.
id },
      });

      await prisma.channel.update({
        where: { id: input.channelId },
        data: { memberCount: { increment: 1 } },
      });

      // System message
      await prisma.message.create({
        data: { channelId: input.channelId, authorId: ctx.use
r.id, content: "joined the channel", type: "system" },
      });

      return member;
    }),

  // Leave a channel
  leave: protectedProcedure
    .input(z.object({ channelId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      await prisma.channelMember.delete({
        where: { channelId_userId: { channelId: input.channel
Id, userId: ctx.user.id } },
      });
      await prisma.channel.update({
        where: { id: input.channelId },
        data: { memberCount: { decrement: 1 } },
      });
      await prisma.message.create({
        data: { channelId: input.channelId, authorId: ctx.use
r.id, content: "left the channel", type: "system" },
      });
      return { success: true };
```

```
    }),

  // Invite user to channel
  invite: protectedProcedure
    .input(z.object({ channelId: z.string().uuid(), userId:
z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const member = await prisma.channelMember.create({
        data: { channelId: input.channelId, userId: input.use
rId },
      });
      await prisma.channel.update({ where: { id: input.channe
lId }, data: { memberCount: { increment: 1 } } });
      await prisma.message.create({
        data: { channelId: input.channelId, authorId: ctx.use
r.id, content: `invited a member to the channel`, type: "syst
em" },
      });
      return member;
    }),

  // Update channel settings
  update: protectedProcedure
    .input(z.object({
      channelId: z.string().uuid(),
      displayName: z.string().optional(),
      description: z.string().optional(),
      topic: z.string().optional(),
      icon: z.string().optional(),
      color: z.string().optional(),
      postingPermission: z.string().optional(),
    }))
    .mutation(async ({ input }) => {
      const { channelId, ...data } = input;
      return prisma.channel.update({ where: { id: channelId
}, data });
```

```
    }),

  // Archive channel
  archive: protectedProcedure
    .input(z.object({ channelId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.channel.update({ where: { id: input.chann
elId }, data: { isArchived: true } });
    }),

  // Star/unstar channel (for sidebar ordering)
  toggleStar: protectedProcedure
    .input(z.object({ channelId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const member = await prisma.channelMember.findUnique({
        where: { channelId_userId: { channelId: input.channel
Id, userId: ctx.user.id } },
      });
      if (!member) throw new Error("Not a member");
      return prisma.channelMember.update({
        where: { id: member.id },
        data: { isStarred: !member.isStarred },
      });
    }),

  // Set notification preference for channel
  setNotificationPref: protectedProcedure
    .input(z.object({
      channelId: z.string().uuid(),
      pref: z.enum(["all", "mentions_only", "nothing"]),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.channelMember.update({
        where: { channelId_userId: { channelId: input.channel
Id, userId: ctx.user.id } },
        data: { notificationPref: input.pref },
```

```
      });
    }),

  // Mute/unmute channel
  toggleMute: protectedProcedure
    .input(z.object({ channelId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const member = await prisma.channelMember.findUnique({
        where: { channelId_userId: { channelId: input.channel
Id, userId: ctx.user.id } },
      });
      if (!member) throw new Error("Not a member");
      return prisma.channelMember.update({
        where: { id: member.id },
        data: { isMuted: !member.isMuted },
      });
    }),
});
```

FEATURE: DM / GROUP CONVERSATIONS

tRPC Router: `src/server/routers/conversations.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import crypto from "crypto";

function hashParticipants(ids: string[]): string {
  const sorted = [...ids].sort();
  return crypto.createHash("sha256").update(sorted.join
(",")).digest("hex");
}
```

```
export const conversationsRouter = router({
  // List user's DM conversations
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      const memberships = await prisma.conversationMember.fin
dMany({
        where: { userId: ctx.user.id },
        include: {
          conversation: {
            select: {
              id: true, type: true, name: true, icon: true,
              participantIds: true, lastMessageAt: true, last
MessagePreview: true,
            },
          },
        },
        orderBy: { conversation: { lastMessageAt: "desc" } },
      });

      return memberships.map(m => ({
        ...m.conversation,
        isMuted: m.isMuted,
      }));
    }),

  // Create or get existing DM conversation
  createOrGet: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      participantIds: z.array(z.string().uuid()).min(1).max
(8), // other participants (not including self)
    }))
    .mutation(async ({ input, ctx }) => {
      const allParticipants = [...new Set([ctx.user.id, ...in
put.participantIds])];
```

```
      if (allParticipants.length < 2) throw new Error("Need a
t least 2 participants");
      if (allParticipants.length > 9) throw new Error("Maximu
m 9 participants in a group");

      const hash = hashParticipants(allParticipants);
      const type = allParticipants.length === 2 ? "dm" : "gro
up";

      // Check for existing conversation
      const existing = await prisma.conversation.findUnique({
        where: { workspaceId_participantHash: { workspaceId:
input.workspaceId, participantHash: hash } },
      });
      if (existing) return existing;

      // Create new conversation
      const conversation = await prisma.conversation.create({
        data: {
          workspaceId: input.workspaceId,
          type,
          participantIds: allParticipants,
          participantHash: hash,
          createdBy: ctx.user.id,
        },
      });

      // Add all participants as members
      await prisma.conversationMember.createMany({
        data: allParticipants.map(userId => ({
          conversationId: conversation.id,
          userId,
        })),
      });

      return conversation;
```

```
    }),

  // Get messages in a conversation
  getMessages: protectedProcedure
    .input(z.object({
      conversationId: z.string().uuid(),
      limit: z.number().default(50),
      cursor: z.string().uuid().optional(),
    }))
    .query(async ({ input, ctx }) => {
      // Verify membership
      const member = await prisma.conversationMember.findUniq
ue({
        where: { conversationId_userId: { conversationId: inp
ut.conversationId, userId: ctx.user.id } },
      });
      if (!member) throw new Error("Not a participant");

      const messages = await prisma.directMessage.findMany({
        where: { conversationId: input.conversationId, delete
dAt: null },
        orderBy: { createdAt: "desc" },
        take: input.limit + 1,
        ...(input.cursor ? { cursor: { id: input.cursor }, sk
ip: 1 } : {}),
        include: { attachments: true },
      });

      const hasMore = messages.length > input.limit;
      return {
        messages: messages.slice(0, input.limit).reverse(),
        hasMore,
        nextCursor: hasMore ? messages[input.limit - 1]?.id :
undefined,
      };
    }),
```

```
  // Send DM (via Socket.IO for real-time, but also available
as tRPC)
  sendMessage: protectedProcedure
    .input(z.object({
      conversationId: z.string().uuid(),
      content: z.string().min(1).max(40000),
      contentHtml: z.string().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      const msg = await prisma.directMessage.create({
        data: {
          conversationId: input.conversationId,
          authorId: ctx.user.id,
          content: input.content,
          contentHtml: input.contentHtml,
        },
      });

      await prisma.conversation.update({
        where: { id: input.conversationId },
        data: {
          lastMessageAt: new Date(),
          lastMessagePreview: input.content.slice(0, 200),
          messageCount: { increment: 1 },
        },
      });

      return msg;
    }),

  // Update group name/icon
  updateGroup: protectedProcedure
    .input(z.object({
      conversationId: z.string().uuid(),
      name: z.string().optional(),
```

```
      icon: z.string().optional(),
    }))
    .mutation(async ({ input }) => {
      const { conversationId, ...data } = input;
      return prisma.conversation.update({ where: { id: conver
sationId }, data });
    }),
});
```

KEY UI COMPONENTS

Messaging Layout: `src/components/messaging/messaging-layout.tsx`

Build a 3-column layout:

- **Left (260px)**: Channel sidebar with sections: Starred, Channels, DMs, + browse button
- **Center (flex)**: Channel header + message list + composer
- **Right (380px, collapsible)**: Thread panel or channel details

Channel Sidebar: `src/components/messaging/channel-sidebar.tsx`

Structure:

```
┌──────────────────────────────┐
│ 🔍 Search channels (Ctrl+K) │
├──────────────────────────────┤
│ ⭐ STARRED                    │
│   # general        (3) ●     │
│   # engineering     ●        │
├──────────────────────────────┤
│ 📢 CHANNELS                   │
│   # product               ●  │
│   # design                   │
│   # random                   │
```

```
│   [+ Browse channels]          │
├──────────────────────────────┤
│ 💬 DIRECT MESSAGES             │
│   🟢 Alice Chen      (2) ●   │
│   🟡 Bob Smith               │
│   🔴 Carol (DND)             │
│   Team Chat (3 people)        │
│   [+ New message]             │
└──────────────────────────────┘
```

Each item shows: icon, name, unread badge, online dot. Bold if unread. Click to navigate.

Message Composer: `src/components/messaging/message-composer.tsx`

Build with:

- Rich text toolbar: **B** *I* ~~S~~ `code` > quote • list 1. list
- @mention autocomplete: Type "@" → dropdown of workspace members with search, avatar, status
- #channel link: Type "#" → dropdown of channels
- Emoji picker: Click 😊 button → categorized emoji grid with search
- File upload: 📎 button + drag-drop + Ctrl+V paste
- Slash commands: Type "/" → command list dropdown
- Send button + Ctrl+Enter to send
- Typing indicator shown below composer
- Scheduled send: Click ▾ on send button → "Schedule for later" → date/time picker

Message Item: `src/components/messaging/message-item.tsx`

Structure per message:

```
[Avatar] [Name] [Timestamp]          [hover: 😊 💬 📌 ⋯]
         [Message content - supports markdown, code blocks, i
mages]
         [Reactions: 👍2  🎉1  ❤3]
         [Thread: 5 replies · Last reply 2h ago · 👤👤👤]
```

Hover actions bar (appears on hover):

- 😊 Quick react (shows top 5 frequently used emojis)
- 💬 Reply in thread
- 📌 Pin message
- 🔖 Bookmark
- ↗️ Forward
- ✏️ Edit (only for own messages)
- 🗑️ Delete (own messages or admin)
- ⋯ More menu

Slash Commands (built-in):

```
/task [title] — Create a task in linked project database
/remind [who] [when] [what] — Set a reminder
/poll [question] | [option1] | [option2] | ... — Create a pol
l
/code [language] — Insert a code block
/giphy [search] — Search and insert a GIF
/shrug — Append ¯\_(ツ)_/¯
/status [emoji] [text] — Set your status
/dnd [duration] — Set Do Not Disturb
/mute — Mute current channel
/invite @user — Invite user to current channel
/topic [text] — Set channel topic
```

```
/ai summarize — AI summarize recent messages
/ai catch-up — AI catch-up since last visit
```

EDGE CASES — SPRINT 2

```
1. Create channel with name that exists → "Channel #name alre
ady exists"
2. Join private channel directly → "Cannot join — must be inv
ited"
3. Leave last channel (owner) → "Transfer ownership before le
aving" or auto-assign next admin
4. Create DM with yourself → Prevent, show "Cannot message yo
urself"
5. Create DM with same person twice → Return existing convers
ation (dedup via hash)
6. Group DM with >9 people → "Maximum 9 participants"
7. @mention in announcement channel where user can't post → s
till notifies
8. Type >40,000 chars → "Message too long" validation
9. File upload >25MB → "File too large (max 25MB)"
10. Emoji not found → Show text :emoji_name: instead of break
ing
```

VERIFICATION — MESSAGING SPRINT 2

```
□ Create channel "test-channel" → appears in sidebar
□ Browse channels → see all public channels with join button
□ Join channel → badge shows, messages load
□ Leave channel → removed from sidebar
□ Star channel → moves to Starred section
□ Mute channel → icon changes, no notifications
□ Set channel topic → appears in header
□ Start DM with another user → conversation created
```

```
☐ Create group DM (3 people) → group appears in DM section
☐ Set custom group name → name updates
☐ Message composer: type **bold** → renders bold
☐ Type @alice → autocomplete shows, select → mention inserted
☐ Type #general → channel link autocomplete works
☐ Click emoji picker → categories, search, click to insert
☐ Drag file onto composer → upload starts → preview shown
☐ Type /task Buy groceries → task created (if linked project)
☐ Ctrl+Enter sends message
☐ Hover on message → action bar appears
☐ Click edit → inline edit mode → save
☐ Click delete → confirmation → message replaced with "delete
d"
☐ Click react 👍 → reaction appears below message
☐ Unread badge shows correct count
☐ Mobile: sidebar collapses, swipe to navigate
```

Once all checks pass, proceed to Messaging Sprint 3.